Proyecto #2 Parte 2 — HPC con OpenMPI colectivo

Integrantes: Iris Ayala (23965), Gabriel Bran (23590), Anggie Quezada (23643)

Fase de Validación: Descifrado Cruzado Local

Texto elegido del foro

Equipo que eligieron	Texto	Palabra objetivo
Kevin Villagrán / Joel Jaquez (grupo publicado en el foro como "minecraft biomas")	Texto sobre Open MPI, 2136 bytes / 267 bloques DES, publicado en base64	Open MPI es una

Adaptación del programa

El programa original (bruteforce_fix.c) cifraba el texto él mismo con la llave recibida por argumento y luego la buscaba, lo cual no sirve para descifrar un texto ya cifrado por otro equipo. Se agregó un modo nuevo que decodifica y lee directamente el ciphertext en base64 de otro equipo, sin modificar el comportamiento existente usado en todas las mediciones de la Parte 1:

```
mpirun -np N bruteforce_fix --b64 <archivo.b64> "<frase clave>"
[block|cyclic] [bits]
```

Búsqueda de la llave privada

Antes de obtener la llave por confirmación directa del equipo, se ejecutó una búsqueda por fuerza bruta a ciegas sobre el espacio completo de DES (2^56) con reparto cíclico y 16 procesos locales, cubriendo sin éxito las primeras ~300 millones de llaves. Se descartó también que el otro equipo hubiera usado el armado de llave original, se probó explícitamente en el rango [0, 16 000 000) y la palabra objetivo no apareció.

Se obtuvo la llave real por confirmación directa del equipo autor del texto: **18014398509481984** (= 2^54). Se verificó de dos formas independientes antes de darla por buena:

1. **Verificación propia:** ejecutar bruteforce --b64 con esa llave exacta decodifica el texto completo y legible sobre Open MPI (ver [IMG-7]).
2. **Verificación criptográfica cruzada:** se recompiló el propio cifrar.c del equipo autor y se cifró su propio texto con esa llave; el resultado coincide byte a byte en 266 de 267 bloques DES-ECB con el ciphertext que publicaron (el único bloque distinto es una diferencia de fin de línea en la copia local del texto, no de la llave). Una coincidencia así en bloques ECB independientes es estadísticamente imposible con una llave incorrecta.

Log de tiempo y speedup (punto de referencia Ts)

Ejecutando localmente (una sola máquina, 4 procesos, espacio completo 2^56, reparto block):

Corrida	Procesos	Nodos	Tiempo	Speedup	Eficiencia
Local (Ts)	4	1	0.017601 s	1.00	1.00

El texto plano se recupera completo y correcto (ver [IMG-7]). Este tiempo es el punto de referencia Ts que se usa en la Sección V/VI para calcular el speedup al distribuir el trabajo en el clúster físico.

Virtualización del Entorno: Preparación con Docker

Se redactó un Dockerfile en la raíz del proyecto cumpliendo las especificaciones obligatorias:

1. **Dependencias base:** imagen oficial ubuntu:22.04, con build-essential, openmpi-bin, libopenmpi-dev, openssh-server instalados vía apt-get. Se agregó además libssl-dev, necesaria porque el código enlaza con -lcrypto (el shim compat/rpc/des_crypt.h usa el DES de OpenSSL, ya que Ubuntu 22.04/glibc 2.35 eliminó las funciones de Sun RPC).
2. **SSH sin intervención:** PermitRootLogin yes en /etc/ssh/sshd_config y StrictHostKeyChecking no en /etc/ssh/ssh_config, con una llave RSA sin passphrase generada y autoautorizada en el build.
3. **Ejecución como root en OpenMPI:** ENV OMPI_ALLOW_RUN_AS_ROOT=1 y ENV OMPI_ALLOW_RUN_AS_ROOT_CONFIRM=1.

Tabla de validación

Validar	**Esperado**	**¿Funcionó?**	**Qué se tuvo que modificar o hacer para lograrlo**
docker run --rm mpi-cluster-image mpicc --version	gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0	Sí	Nada
docker run --rm mpi-cluster-image mpirun -np 2 echo "Proceso MPI ejecutado"	Lo imprime 2 veces	Sí	Las variables OMPI_ALLOW_RUN_AS_ROOT; sin ellas OpenMPI se niega a correr como root
docker run -it --rm mpi-cluster-image /bin/bash	Entra a la terminal	Sí	Nada
service ssh start	Arranca sshd	Sí	Nada
ssh-keygen -t rsa -N "" -f ~/.ssh/id_rsa + copiar a authorized_keys	Genera una llave y se prueba local	Sí	Ya viene generada y autoautorizada desde el build
ssh localhost echo "Conexion SSH sin contrasena exitosa"	Imprime el mensaje	Sí	PermitRootLogin yes y StrictHostKeyChecking no en el Dockerfile

[IMG-1] — captura de los 6 comandos de esta tabla corridos en secuencia.

Despliegue de Red Física y Consideraciones de Topología

Por qué se cambió de --network host a Tailscale

Se siguió primero la instrucción literal del enunciado (docker run --network host). **No funcionó en Windows**: aun con Docker Desktop ≥4.34, "Enable host networking" activo ([IMG-2]), el firewall configurado, y WSL2 en modo mirrored, el contenedor seguía viendo solo IPs internas de la VM de Docker (192.168.65.x/172.x), nunca la IP real del router. Se confirmó que es una limitación de diseño: en Linux, --network host comparte literalmente el namespace de red del host; en Docker Desktop (Windows/Mac), el "host" que ve el contenedor es la VM interna de Docker, no la máquina física — verificado de forma concluyente usando un motor Docker nativo dentro de WSL2 (ahí sí funcionó como en Linux real), pero se descartó por ser frágil de coordinar entre varias personas con sistemas distintos.

Se evaluó también Radmin VPN, descartada porque el contenedor no ve su adaptador virtual y no existe cliente para Mac.

Se adoptó Tailscale (VPN mesh basada en WireGuard) como mecanismo equivalente para lograr el mismo objetivo del enunciado (que los contenedores de distintas computadoras se vean entre sí), sin depender de --network host: cada contenedor corre su propio demonio tailscaled, con --cap-add NET_ADMIN --device /dev/net/tun, y recibe una IP fija 100.x.x.x igual en cualquier sistema operativo o red física. Esto cumple el mismo rol que cumpliría la subred física del router en las instrucciones del MCA de OpenMPI, solo que con tailscale0 como interfaz en vez de una subred LAN.

Problemas de red enfrentados (troubleshooting)

#	Problema	Causa raíz	Solución
1	--network host no exponía la IP real en Windows	Docker Desktop virtualiza su propio namespace de red; "Enable host networking" solo expone puertos a localhost	Se descartó esta vía; se adoptó Tailscale
2	Radmin VPN como alternativa	El contenedor no ve el adaptador de Radmin; sin cliente para Mac	Descartado
3	Build de la imagen con Tailscale falla con error de certificado SSL (unable to get local issuer certificate) en la red de la universidad	El firewall institucional (Fortinet) intercepta TLS (man-in-the-middle) sobre tráfico HTTPS saliente, incluida la descarga del instalador de Tailscale ([IMG-3])	Construir la imagen desde otra red (datos móviles); funciona sin problema en la red de la universidad una vez construida

#	Problema	Causa raíz	Solución
4	El tráfico de Tailscale en sí (no solo la descarga) se bloqueaba en la red de la universidad	Mismo firewall bloqueando el túnel WireGuard/DERP en tiempo real ([IMG-4])	Operar el clúster desde una red fuera del campus (hotspot)
5	La IP física cambió de sesión en sesión (hotspot ↔ red universidad)	DHCP reasigna otra subred al reconectar	Con Tailscale, la IP 100.x no cambia aunque cambie la red física subyacente
6	Mismo nombre de contenedor corriendo en dos motores Docker distintos a la vez (motor nativo de WSL vs. Docker Desktop)	Son dos daemons y almacenes de imágenes completamente separados en la misma máquina Windows	Verificar siempre con docker ps -a en qué motor está realmente el contenedor antes de depurar
7	Intercambio manual de la llave pública SSH se corrompía repetidas veces (espacios insertados en medio del bloque base64)	La app de chat usada para pasar la llave reflowea el texto largo e inserta espacios al mostrarlo	Se dejó de pegar la llave a mano: se transmitió como archivo versionado en el repositorio (git pull + docker cp)
8	Hostname con guion bajo (mpi_worker) hace que orted remoto truene con segfault	Bug conocido de OpenMPI 4.1.2 con nombres de host inválidos	Usar hostnames con guion normal: nodo-maestro, nodo-worker1
9	Puerto 22 ocupado / sshd no arrancado en el contenedor	El CMD/entrypoint de la imagen no arranca sshd automáticamente	Script nodo_start.sh que arranca service ssh start al inicio de cada nodo
10	Búsqueda a ciegas por fuerza bruta de la llave del otro equipo (2^56) sin ninguna pista	No se tenía ningún indicio del orden de magnitud de la llave	Preguntar directamente al otro equipo — confirmaron 2^54, ahorrando horas de cómputo (ver Sección III)

Paso A: Sincronización de llaves SSH inter-nodo

Desde el contenedor del Nodo Maestro se generó la llave pública (ssh-keygen ya la trae el build) y se copió al authorized_keys del contenedor del Nodo Worker, usando las IPs de Tailscale de cada uno en vez de IPs de router:

```
ssh_keys.sh pub                       # en el Maestro, imprime la llave
pública
ssh_keys.sh add "ssh-rsa AAAA..."     # en el Worker, la autoriza
ssh_keys.sh test 100.x.x.x            # en el Maestro, prueba sin contraseña
```

Resultado: SSH sin contrasena OK en nodo-worker1 (100.73.84.34 ...) — **[IMG-5]**.

Paso B: Ejecución del ataque distribuido

Con el hostfile configurado con las IPs de Tailscale de cada nodo y tailscale0 como interfaz en los parámetros MCA (equivalente a la subred física del router que pide el enunciado):

```
mpirun --hostfile /app/cluster_hosts \
  --mca btl_tcp_if_include tailscale0 --mca oob_tcp_if_include tailscale0 \
  --display-map -np 8 /app/bruteforce --b64
/app/cruzado/grupo_minecraft/texto_cifrado.b64 \
  "Open MPI es una" block 56
```

Discusión de Resultados y Conclusiones

1. ¿Por qué OpenMPI se congela indefinidamente sin --mca btl_tcp_if_include?

Cuando MPI arranca, cada proceso anuncia las direcciones IP de todas sus interfaces de red para que los demás nodos sepan cómo contactarlo (canal "out-of-band", OOB, y luego el canal de datos BTL/TCP). Si no se limita la búsqueda a una subred, OpenMPI incluye interfaces virtuales como docker0, veth* o adaptadores de VPN/VirtualBox. Esas interfaces existen en todas las máquinas pero casi nunca están en la misma subred entre una máquina y otra (docker0 suele ser 172.17.0.1/16 en cualquier instalación de Docker, pero no son la misma red física entre dos computadoras). El proceso remoto intenta conectarse por esa IP, el paquete nunca llega (no hay ruta real o cae en una NAT distinta), y como TCP reintenta en vez de fallar inmediatamente, el programa se queda esperando una respuesta que nunca llega — sin mensaje de error, porque desde el punto de vista de MPI sigue "intentando conectar". --mca btl_tcp_if_include / oob_tcp_if_include fuerzan a MPI a anunciar y usar solo una interfaz conocida y alcanzable entre todos los nodos (la subred del router, o en nuestro caso tailscale0), eliminando la ambigüedad.

2. Problemas de red enfrentados

Ver la tabla completa de 10 problemas en la Sección V. En resumen: la limitación de diseño de Docker Desktop con --network host en Windows, un firewall institucional (Fortinet) que interceptaba TLS y bloqueaba el propio tráfico de Tailscale, cambios de IP física entre sesiones, confusión entre dos motores Docker distintos corriendo en la misma máquina, y la corrupción repetida de una llave SSH al transmitirla por una app de chat que reflowea texto largo.

3. Tiempo local vs. clúster físico

Se midió en dos escenarios distintos, con la llave real del grupo cruzado (2^54) y con la llave propia de la Parte 1 (4206649), para mostrar cómo el resultado depende de dónde cae la llave en el espacio de búsqueda:

Escenario	Ts (local)	Tp (clúster, 2 nodos)	Speedup	Eficiencia
Llave real del grupo cruzado (2^54, cae al borde de un rango)	0.017601 s	1.460849 s	0.012 (más lento)	0.0015
Llave propia (4206649, cae en medio del espacio)	18.183062 s	10.938858 s	1.66	0.21

Con la llave real, el clúster fue ~83 veces más lento, no más rápido. La llave cayó justo al inicio del rango de un proceso tanto local como distribuido (encontrada en el primer intento, tried=0 en ambos casos), así que el cómputo real es insignificante; lo que domina en la corrida distribuida es el arranque del proceso remoto por SSH y la sincronización a través del relay DERP de Tailscale (~200-300 ms de latencia medidos con tailscale ping, nunca hubo conexión P2P directa). Con la llave propia, en cambio, sí hubo mejora real porque el trabajo de búsqueda es sustancial y se reparte de forma efectiva entre los procesos de ambos nodos. Esto confirma, en un entorno de red real, el mismo patrón que ya se había documentado en la Parte 1 (con reparto en bloques, el speedup depende fuertemente de dónde cae la llave, no solo de cuántos procesos/nodos se usen).

4. ¿Qué pasa con 32 procesos MPI en una máquina de 4 núcleos físicos?

Es oversubscription severa: el sistema operativo tiene que repartir 32 procesos activos entre solo 4 núcleos lógicos mediante cambios de contexto constantes. Cada cambio de contexto tiene un costo fijo (guardar/restaurar registros, invalidar cachés L1/L2 del núcleo), y con DES —una carga intensiva en CPU sin esperas de I/O— ese costo no se esconde detrás de ningún tiempo muerto. El resultado no es un error, sino un rendimiento mucho peor que con 4 procesos: el tiempo total aumenta porque la mayor parte se gasta cambiando entre procesos en vez de calculando, y además compiten entre sí por las mismas cachés, degradando también el rendimiento individual de cada uno. Es la misma razón por la que el hostfile debe declarar max_slots igual al número real de núcleos físicos, nunca más.

5. Vulnerabilidades y retos logísticos de HPC con hardware heterogéneo

- **Superficie de ataque de SSH sin restricciones.** Para que MPI funcione entre nodos se necesita PermitRootLogin yes y acceso sin contraseña — exactamente lo que cualquier guía de seguridad recomienda evitar. Aceptable en un contenedor efímero de laboratorio, pero inaceptable en producción sin aislar esas llaves y esa configuración a una red privada y confiable.
- **Confianza implícita entre nodos.** Una vez que el maestro puede ejecutar comandos sin contraseña en los workers, cualquier nodo comprometido compromete a todo el clúster.
- **Heterogeneidad real de hardware y SO.** Windows, Mac y Linux se comportan de forma distinta frente a redes virtuales, firewalls y virtualización de contenedores — lo que

funciona en una máquina no garantiza nada en otra, como se vio con Docker Desktop. Un administrador de HPC real tiene que anticipar y documentar estas diferencias, no asumir homogeneidad.

- **Dependencia de servicios de terceros.** Usar una VPN mesh como Tailscale resuelve el problema de red, pero añade una dependencia externa (su propio servicio de control y, a veces, sus relays DERP) que no existe si todos los nodos están en la misma LAN real.
- **Coordinación logística entre personas.** Sincronizar llaves SSH, IPs y versiones de la imagen entre varias personas en tiempo real, cada una con su propia conexión y sistema, fue en la práctica más difícil que el código en sí — y a escala real (decenas o cientos de nodos) ese problema solo crece.

¿Lograron el objetivo? ¿Qué fue lo más difícil? ¿Qué se haría distinto?

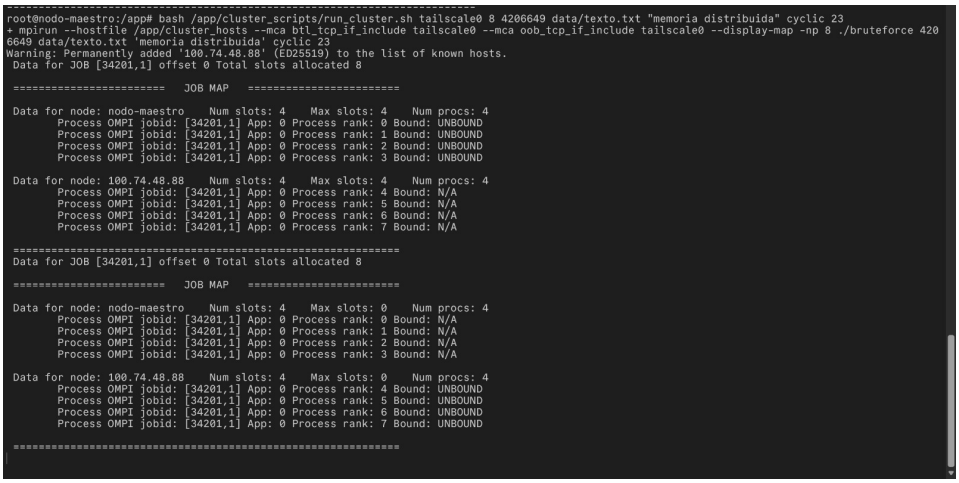
Sí se logró el objetivo completo: se adaptó el programa para leer el ciphertext de otro equipo, se confirmó la llave real con doble verificación, y se corrió la comparación Ts/Tp en un clúster físico real con dos máquinas distintas conectadas por Tailscale.

Lo más difícil no fue el código, fue la infraestructura de red: Docker Desktop en Windows resultó no compartir la interfaz física con el contenedor (un límite de diseño, no un error de configuración), el firewall de la universidad interceptaba TLS y bloqueaba el tráfico de Tailscale, la IP física cambió varias veces de sesión en sesión, y la sincronización manual de la llave SSH pública se corrompió repetidas veces al pasarla por una app de chat.

Lo que se haría distinto: pedir desde el principio el orden de magnitud de la llave del equipo cruzado (o al menos confirmar si usan el armado de llave original o corregido) antes de lanzar una búsqueda a ciegas sobre 2^56 — se gastaron varios minutos de cómputo en balde que se habrían evitado con una pregunta directa.

Lo que más gustó: ver que el mismo binario y el mismo código, sin cambios, corrieron sin problema en máquinas con sistemas operativos distintos gracias a

Imágenes



```
--------------------------------------------------------------------------
root@nodo-maestro:/app# bash /app/cluster_scripts/run_cluster.sh tailscale0 8 4206649 data/texto.txt "memoria distribuida" cyclic 23
+ mpirun --hostfile /app/cluster_hosts --mca btl_tcp_if_include tailscale0 --mca oob_tcp_if_include tailscale0 --display-map -np 8 ./bruteforce 420
6649 data/texto.txt 'memoria distribuida' cyclic 23
Warning: Permanently added '100.74.48.88' (ED25519) to the list of known hosts.
 Data for JOB [34201,1] offset 0 Total slots allocated 8

 ========================   JOB MAP   ========================

 Data for node: nodo-maestro    Num slots: 4    Max slots: 4    Num procs: 4
        Process OMPI jobid: [34201,1] App: 0 Process rank: 0 Bound: UNBOUND
        Process OMPI jobid: [34201,1] App: 0 Process rank: 1 Bound: UNBOUND
        Process OMPI jobid: [34201,1] App: 0 Process rank: 2 Bound: UNBOUND
        Process OMPI jobid: [34201,1] App: 0 Process rank: 3 Bound: UNBOUND

 Data for node: 100.74.48.88    Num slots: 4    Max slots: 4    Num procs: 4
        Process OMPI jobid: [34201,1] App: 0 Process rank: 4 Bound: N/A
        Process OMPI jobid: [34201,1] App: 0 Process rank: 5 Bound: N/A
        Process OMPI jobid: [34201,1] App: 0 Process rank: 6 Bound: N/A
        Process OMPI jobid: [34201,1] App: 0 Process rank: 7 Bound: N/A

 =============================================================
 Data for JOB [34201,1] offset 0 Total slots allocated 8

 ========================   JOB MAP   ========================

 Data for node: nodo-maestro    Num slots: 4    Max slots: 0    Num procs: 4
        Process OMPI jobid: [34201,1] App: 0 Process rank: 0 Bound: N/A
        Process OMPI jobid: [34201,1] App: 0 Process rank: 1 Bound: N/A
        Process OMPI jobid: [34201,1] App: 0 Process rank: 2 Bound: N/A
        Process OMPI jobid: [34201,1] App: 0 Process rank: 3 Bound: N/A

 Data for node: 100.74.48.88    Num slots: 4    Max slots: 0    Num procs: 4
        Process OMPI jobid: [34201,1] App: 0 Process rank: 4 Bound: UNBOUND
        Process OMPI jobid: [34201,1] App: 0 Process rank: 5 Bound: UNBOUND
        Process OMPI jobid: [34201,1] App: 0 Process rank: 6 Bound: UNBOUND
        Process OMPI jobid: [34201,1] App: 0 Process rank: 7 Bound: UNBOUND

 =============================================================
```

```
root@nodo-maestro:/app# mpicc --version
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

root@nodo-maestro:/app# mpirun -np 2 echo


root@nodo-maestro:/app# mpirun -np 2 echo ...
...
...
root@nodo-maestro:/app# mpirun -np 2 echo ...
...
...
root@nodo-maestro:/app# service ssh start
 * Starting OpenBSD Secure Shell server sshd                                                                    [ OK ]
root@nodo-maestro:/app# ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa): exit
Enter passphrase (empty for no passphrase):

root@nodo-maestro:/app# ssh localhost echo ...
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
...
root@nodo-maestro:/app#
```

```
David@LAPTOP-ETV994DU ~ Documents 2026 S2 PARALELA BRAM proyecto2_paralela  test
❯ docker exec -it mpi_ts bash
root@nodo-worker1:/app# bash /app/cluster_scripts/nodo_start.sh
 * Starting OpenBSD Secure Shell server sshd                                                                    [ OK ]
hostname : nodo-worker1
IPs      : 172.17.0.2 100.74.48.88 fd7a:115c:a1e0::494d:3059
nucleos  : 12
binario  : -rwxr-xr-x 1 root root 21920 Oct 27 22:12 /app/bruteforce
root@nodo-worker1:/app# bash /app/cluster_scripts/ssh_keys.sh add "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDDuAixsV67T/Fx/fruIMSoOXjl+jw0zcjMnhc4ZuAUboZof347g
Lyy9rrO4kxYLL2pEHq10cIqtb14lhHdfiP9gDAqRA3WNuOmmdNJzIF75CmgchZbO9B9IpV2EK3D4EOFj0OFSuLC0Jyid6NB/rLMMJaHhocIFOq1t2O0/uSU4DOovI53A0PDQohMPCSZ6vGd5RH5OxJEl5y47
TuxnMMVslPEL0Nste2LetatJq8vCMFS2lp7TZ2rneIK+q5X+hk/vfS6UWXptVqlbG+f+fyjDxpmyv4JY2OOC61cNgIg4vjz5kJLOgR0uqwBbzTAVtUvq3vlorpOJe8M1N0Ul2yBY2uFXUMDMGtjvbKNgp/oP
udL6b3ZwGJTxgq2jNmTZL7jQvI6GIkdDiuynBx+Ou2bpbuwyMAIY15AxuAuqrrTvb/a7m2bM0amW4j/FFRUxxovr0imU0lm1I3ltKLT1cBDgMIIgw2c8PohLvnaAC6AbvmCjQXPqV+P5zJwp3Nppts= root
@buildkitsandbox"
Llave autorizada.
root@nodo-worker1:/app#
```